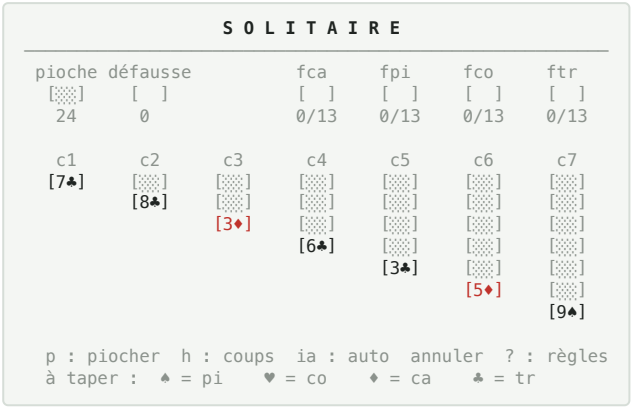
Le Solitaire dossier de soutenance, version courte

Mattias Mathevon et Jules Damour · L3 MIAGE
Soutenance du vendredi 6 novembre 2026

1. Les structures de données

Le sujet interdit les classes : tout le jeu tient dans des **textes, des listes, des dictionnaires et des tuples**.

- **Une carte est un texte** : sa valeur (`a`, `2` à `10`, `v`, `d`, `r`) puis sa famille en deux lettres (`pi`, `co`, `ca`, `tr`). Ex : `"5ca"` est le 5♦. `carte[:-2]` donne la valeur, `carte[-2:]` la famille.
- **`piles`** : un dictionnaire de 13 listes. `"p"` la pioche, `"d"` la défausse, `"c1"` à `"c7"` les colonnes, `"fca"`, `"fpi"`, `"fco"`, `"ftr"` les fondations. Partout, **la dernière carte de la liste est celle du dessus** : on la lit avec `[-1]`, on la retire avec `pop()`, on pose avec `append()`.
- **`cachees`** : pour chaque colonne, combien de cartes sont face cachée (toujours en bas), ex : `{"c1": 0, "c2": 1, ..., "c7": 6}`. Une carte est cachée si sa position est plus petite que ce nombre.
- **`index`** : pour chaque carte, le nom de sa pile. Quand le joueur tape `5ca-c4`, `index["5ca"]` donne tout de suite le départ, sans chercher dans les 13 piles. Il faut le mettre à jour à chaque déplacement et à chaque pioche.
- **Le reste** : `nb_coups` et `difficulte` sont des entiers ; la sauvegarde est un dictionnaire écrit en JSON ; pour `annuler`, la partie est copiée avant chaque coup (chaque liste copiée avec `list()`, sinon la copie partagerait les mêmes cartes).



La partie de démonstration au départ : 18 lignes sur 56 caractères (limite : 20 × 60).

Un coup pas à pas, `5ca-c4`. `verif_coup()` renvoie le tuple `("5ca", "c4")`. `index["5ca"]` donne `"c6"`, sa position est 5, le morceau à déplacer est `piles["c6"][5:]` (la carte et toutes celles posées dessus). Après les vérifications, `del` retire le morceau, `extend()` le pose sur c4, `index["5ca"]` devient `"c4"`. Il ne reste que des cartes cachées en c6 : `cachees["c6"]` passe de 5 à 4 et le 3♥ se retourne.

2. Les commandes de jeu et les menus

Le menu de départ : 1 nouvelle partie (paquet mélangé avec `random.shuffle()` puis distribué), 2 reprendre la sauvegarde, 3 choisir le niveau, 4 quitter, 5 partie de démonstration (toujours la même donne). Le menu redemande tant que la réponse n'est pas entre 1 et 5.

Saisie	Effet	Compte comme un coup
`carte-pile`	déplace la carte et toutes celles posées dessus, ex : `5ca-c4` ; pas besoin de dire d'où elle part	oui
`p`	pioche 1, 2 ou 3 cartes selon le niveau ; pioche vide : la défausse retourne dans la pioche	non
`h`	affiche les coups possibles, du plus utile au moins utile	non
`ia`	l'ordinateur joue tout seul jusqu'à gagner ou être bloqué	oui, chacun de ses coups
`annuler`	revient avant le dernier coup ou la dernière pioche (une fois)	le compteur recule
`sauvegarde`	enregistre la partie dans `sauvegarde.json`	non
`?`	affiche les règles et toutes les commandes	non

Toutes les saisies sont vérifiées, dans cet ordre. Dès qu'une vérification échoue, le jeu dit pourquoi, rien ne bouge et le coup n'est pas compté.

Fonction	Ce qui est contrôlé	Exemple refusé
`verif_coup()`	un seul tiret, une carte et une pile qui existent	`5ca c4`, `5ca-c9`
`verif_carte()`	ni dans la pioche ni face cachée ; celle du dessus dans la défausse ou une fondation ; une seule carte vers une fondation	`dpi-c1`
`verif_colonne()`	colonne vide : un roi seulement ; sinon couleur différente et valeur juste en dessous	`6tr-c2`
`verifier_fondation()`	la bonne famille ; un as sur une fondation vide ; sinon la valeur juste au dessus	`3ca-fca`

3. Comment l'ordinateur propose les coups, et joue tout seul

Trouver les coups (`trouver_coups()`). L'ordinateur ne devine rien : il essaie chacune des 52 cartes (sauf celles de la pioche, face cachée) sur chacune des 11 piles d'arrivée, soit au plus 572 essais. Il garde les coups que le jeu accepterait, en appelant **les mêmes fonctions de vérification que pour le joueur**, avec un paramètre `parler=False` pour qu'elles n'affichent pas leur message de refus. Un coup proposé est donc forcément accepté, et si on corrige une règle, l'ordinateur suit tout seul.

Classer les coups. Chaque coup est rangé dans une liste selon son type, puis les listes sont collées bout à bout avec `+` :

Rang	Type de coup	Pourquoi
1	une carte monte sur sa fondation	c'est le but du jeu
2	la carte part et une carte cachée se retourne	une carte de plus à jouer
3	une carte de la défausse va sur une colonne	la pioche se vide
4	une carte change de colonne et libère celle d'en dessous pour sa fondation	prépare un rang 1
5	toute une colonne part, pour un roi qui attend une place	fait de la place au roi
6	piocher : `p` (sans dire quelle carte va sortir)	une nouvelle carte arrive
7	les autres changements de colonne	la carte change juste de place
8	une carte redescend d'une fondation	la partie recule

Jouer tout seul : `ia`. La boucle du jeu met `auto` à `True`. À chaque tour, au lieu de lire la saisie, elle prend le premier coup de `trouver_coups(..., seulement_utiles=True)` et le joue avec la même fonction `deplacement()` que le joueur (paramètre `coup_ordi`). Deux protections :

- **Pas d'allers-retours** : avec `seulement_utiles=True`, les rangs 7 et 8 ne sont jamais joués. Sinon l'ordinateur pourrait déplacer un 7 noir d'un 8 rouge à l'autre sans fin.
- **Pas de pioche à l'infini** : `nb_tours` compte les fois où il retourne la défausse sans rien jouer d'autre. À la 2e, il a vu toute la pioche : il s'arrête et rend la main au joueur.

Fin automatique, comme dans le vrai solitaire : quand plus aucune carte n'est cachée et que la pioche et la défausse sont vides (`tout_est_retourne()`), la partie est forcément gagnée et l'ordinateur range les cartes dans les fondations.

Résultats. L'ordinateur gagne la partie de démonstration aux trois niveaux (91, 96 et 89 coups). Sur 500 parties au hasard par niveau : 41 % gagnées en facile, 21 % en moyen, 11 % en difficile, sans plantage ni boucle infinie.

4. Les points forts et les particularités du jeu

- **L'ordinateur sait jouer** (`ia`), suit le même classement que `h`, et rend la main au lieu de tourner en rond.
- **Un coup, une saisie** : tout s'écrit `carte-pile`, même pour déplacer plusieurs cartes. **Chaque refus est expliqué.**
- **Annuler** le dernier coup (même une pioche ou un coup de l'ordinateur), **sauvegarder et reprendre** la partie au même niveau.
- **Un plateau compact et lisible** : 18 × 56 caractères, cœurs et carreaux en rouge, cartes cachées `[▒▒]`, compteurs de la pioche et des fondations, écran effacé à chaque coup. Règles avec `?`, animation de victoire.
- **Une partie gagnante** rejouable d'un coup : `partie_gagnante.txt` gagne la démo en 95 coups. On s'en est servi pour vérifier, après chaque modification, qu'on n'avait rien cassé.
- **Travail à deux avec git** : chacun sur sa branche GitHub, fusion par pull request. Le code rendu est regroupé dans un seul fichier, découpé en fonctions et rangé par partie.

5. Les niveaux de jeu et le paramétrage

Le niveau se choisit au menu (choix 3). C'est un simple entier, `difficulte`, égal au **nombre de cartes tirées à chaque pioche** : 1 en facile (par défaut), 2 en moyen, 3 en difficile. `piocher()` s'en sert directement dans sa boucle `while i <= difficulte`, qui s'arrête plus tôt si la pioche se vide.

Comme dans le vrai solitaire, la défausse s'affiche **en éventail** : on voit les cartes de la dernière pioche (`piles["d"][-difficulte:]`), mais seule la dernière est jouable, les autres sont en gris. C'est ce qui rend le niveau difficile plus dur : si on ne joue rien, ce sont toujours les mêmes 8 cartes sur 24 qui arrivent sur le dessus. Le niveau est gardé dans la sauvegarde et affiché dans le menu.

Lancer le jeu : `python3 Solitaire_Mathevon_Damour.py` (sous Windows : `python`). Rejouer la partie gagnante : `python3 Solitaire_Mathevon_Damour.py < partie_gagnante.txt`. Modules utilisés, tous fournis avec Python : `random`, `json`, `os`, `time`.